Federico Bruzzone ✧ Curriculum Vitae

Resident in Magenta (MI), 20013, IT· b. 7th of March 2000
E-mail: federico.bruzzone@unimi.it
E-mail: federico.bruzzone.i@gmail.com
Phone: +39 391 7369214
github.com/FedericoBruzzone
discourse/federicobruzzone

@federicobruzzone
in/federico-bruzzone
@fedebruzzone7
u/FedericoBruzzone
scholar/FedericoBruzzone
0009-0004-6086-8810

Presently a PhD Candidate in Computer Science. I have a strong passion for **Programming Languages** and **Compilers**, and I'm currently diving into **AI compilers**. For more information, just visit my personal **webpage**.

Scientific Publications

Journal Ranked Q1 on Scimago — Federico Bruzzone, Walter Cazzola, and Luca Favini. Prioritizing Configuration Relevance via Compiler-Based Refined Feature Ranking. *Journal of Systems and Software*, page 113096, September 2026 — bib pdf SpringerLink arXiv

Conference Ranked C (National) on CORE — Federico Bruzzone, Lorenzo Capra, Walter Cazzola, and Carlos Olarte. The Minimal Essence of Higher-Order Functions in Maude. In *Proceedings of the 28th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC'26)*. IEEE Computer Society, September 2026 — bib pdf

Conference Ranked B on CORE, CEUR-WS — Lorenzo Capra and Federico Bruzzone. Symmetry preservation in modular rewritable muliformalism models. In *Joint Workshop Proceedings of PNSE'26, PNAS'26, ATAED'26 and PHOCON'26*, volume 4236 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2026 — bib pdf CEUR-WS

Journal Ranked Q1 on Scimago — Federico Bruzzone, Walter Cazzola, Matteo Brancaleoni, and Dario Pellegrino. Sink or swim: Tackling real-time asr at scale. *IEEE Transactions on Audio, Speech and Language Processing*, pages 1–11, 2026 — bib pdf IEEE arXiv

Journal Ranked Q1 on Scimago — Federico Bruzzone, Walter Cazzola, and Luca Favalli. Code less to code more: Streamlining language server protocol and type system development for language families. *Journal of Systems and Software*, page 112554, September 2025 — bib pdf SpringerLink arXiv

Preprints

Federico Bruzzone and Walter Cazzola. A multi-dimensional, per-pass empirical study of the llvm optimization pipeline, 2026 — arXiv

Federico Bruzzone, Walter Cazzola, and Luca Favalli. Generalized software product line extraction, 2026 — arXiv

Federico Bruzzone and Walter Cazzola. Meta-monomorphizing specializations, 2026 — arXiv

Federico Bruzzone, Walter Cazzola, and Luca Favalli. From separate compilation to sound language composition, 2026 — arXiv

Education

2024–Present PhD Candidate in Computer Science at the ADAPT Lab, University of Milan

2022–2024 MSc in Computer Science at University of Milan (*110/110 cum laude*, W. Cazzola, 15/07/2024)
Thesis: Toward a Modular Approach for Type Systems and LSP Generation

2019–2022 BSc in Musical Computer Science at University of Milan (13/10/2022)

2011–2019 Piano and Music Composition at I.S.S.M. Novara Conservatory

2014–2019 Diploma in Computer Science and Telecommunications at E. Alessandrini

Open Source Contributions

2026 Source Matching and Rewriting: Register `artifacts` as a submodule

2026 Source Matching and Rewriting: Add a script to build SMR inside the dev image

2026 Source Matching and Rewriting: Add `lit` tests and a `check-smr` target

2026 Source Matching and Rewriting: Add a regression script over the artifacts cases

2026 MLIR/LLVM: [mlir][vector] Document dimension kinds in `vector.contract` (NFC)

2026 MLIR/LLVM: [mlir][ArmNeon] Add pack/mmt4d path to f32 `linalg.matmul` integration test

2026 Source Matching and Rewriting: Log the triggering conflict when Selector removes a rewrite

2026 Source Matching and Rewriting: Visit CDG candidates in IR order instead of pointer order

2026 Source Matching and Rewriting: Pick the replacement wrapper by the pattern's signature

2026 MLIR/LLVM: [mlir][ArmSME] Make tile allocation order deterministic

2026 MLIR/LLVM: [mlir][ArmSME] Skip tile allocation for functions with no SME tile ops

2026 MLIR/LLVM: [mlir][ArmSME][NFC] Improving the readability of outer product fusion

2026 Source Matching and Rewriting: Reject overlapping matches instead of silently discarding one

2026 MLIR/LLVM: [mlir][linalg] Document and diagnose pack/unpack memref limits

2026 MLIR/LLVM: [mlir][ArmNeon] `matmul-i8.mlir` to `pack-unpack-mmt4d-i8.mlir` + box comments (NFC)

2026 Source Matching and Rewriting: Add `.devcontainer/` to `.gitignore`

2026 MLIR/LLVM: [TailRecElim] Handle discarded-call return conflicts with a shift accumulator

2026 MLIR/LLVM: [mlir][LowerToLLVM] Pass `DataLayoutAnalysis` to `LowerToLLVMOptions` in `ArithToLLVM`, `ControlFlowToLLVM`, and `VectorToLLVM` passes

2026 MLIR/LLVM: [mlir][bufferization] Fix 32-bit overflow in `PromoteBuffersToStack` threshold check

2026 LLVM Lighthouse: Fix failed tests on Darwin

2026 MLIR/LLVM: [mlir][linalg] Add `insert_slice` vectorization tests for leading/trailing dropped dims

2026 MLIR/LLVM: [mlir][linalg] Support non-trailing dropped dims when vectorizing rank-reducing `InsertSliceOp`

2026 MLIR/LLVM: [mlir][SCF] Skip dynamic front-peeling for non-index loops

2026 MLIR/LLVM: [mlir][ArmNeon] Enable native I8MM integration testing on Darwin

2026 MLIR/LLVM: [mlir][vector] Verify non-unit strides on `masked/expand/compress` ops

2026 MLIR/LLVM: [mlir][ArmSME] Enable native ArmSME integration testing on Darwin

2026 MLIR/LLVM: [mlir][ArmSME] Reject non-unit-stride `tile_load/tile_store` memrefs and pass `layout{IdentityLayoutMap}` to matmul tests

2026 MLIR/LLVM: [mlir][ArmNeon] Add `linalg.matmul` e2e tests

2026 MLIR/LLVM: [TailRecElim] Fix miscompile of returns using a non-eliminated recursive call

2026 MLIR/LLVM: [mlir][vector] Verify non-unit strides on `masked/expand/compress` ops (**reverted** by #215224)

2026 MLIR/LLVM: [mlir][ArmNeon] Reject masked contractions in `I8MM/BFMMLA` lowering patterns

2026 MLIR/LLVM: [TailRecElim] Introduce support for shift accumulator optimization

2026 IREE: [GlobalOpt][Flow][Quant] Fix segfault on unlowered `quant.uniform` types

2026 IREE: [Codegen][LLVMCPU] Hoist in-loop stack alloca

2026 IREE: [LLVMCPU] Fix scalable vectorization fallback on SME-only (no +sve) targets

2026 MLIR/LLVM: [mlir][sparse] Avoid vectorizing non-contiguous COO coordinate loads

2026 MLIR/LLVM: [mlir][ArmSVE] Fix comment inconsistency for `pack_lhs` in `ArmSVE/pack-unpack-mmt4d.mlir` (NFC)

2026 MLIR/LLVM: [mlir][vector] add consistent stride verification to masked load/store and gather/scatter ops

2026 MLIR/LLVM: [mlir][ArmSME] fix f64 scalable matmul crashes in `VectorLegalizationPass`

2026 IREE: [LLVMCPU] Fix ArmSME requiring classic SVE, breaking SME-only targets (e.g., Apple Silicon)

2026 IREE: [LLVMCPU] Add SME lowering-strategy tests for f64 and unsupported i8 matmuls

2026 MLIR/LLVM: [mlir][VectorToLLVM] add opt-in `enable-gep-inbounds-nuw` pass flag for `vector.load/store`

2026 MLIR/LLVM: [mlir][MemRefToLLVM] fix incorrect `nuw` on `GEP/mul` when lowering `memref.load/store` with negative strides

2026 MLIR/LLVM: [NFC][mlir][linalg] add `toContractionDimensions` for healthy code reuse

2026 MLIR/LLVM: [mlir][vector] extend `createReadOrMaskedRead/createWriteOrMaskedWrite` with permutation map support

2026 MLIR/LLVM: [mlir][affine] emit `in_bounds` on `transfer_read/write` when statically provable in `affine-super-vectorize`

From 2026 The Eter Programming Language, a new programming language leveraging the MLIR/IREE ecosystem with tensors support.

From 2026 Maintainer of the LLVM Passview, a framework to run an incremental, reproducible study of LLVM optimization passes.

From 2026 Maintainer of the LLVM Pass Template, a C++ template project to quickly create new LLVM passes.

From 2025 Maintainer of the Papers on Compiler Optimizations: Analysis and Transformations, a curated list of scientific publications.

From 2025 Maintainer of scribe, a minimalist, opinionated LaTeX document class for technical writing and presentations.

From 2025 Maintainer of the Tide Compiler, a backend-agnostic IR and compiler framework written in **Rust**.

2025 Generalizing Closeness centrality to weighted networks using Newman method (issue #1384)

2025 Rust: Use the type-level constant value `ty::Value` where needed

2025 Rust: Add `TooGeneric` variant to `LayoutError` and emit `Unknown`

2024 Rust: Report the `note` when specified in `diagnostic::on_unimplemented`

From 2024 Maintainer of the cross-platform tgt project and tdlib-rs TDLib bindings written in **Rust**.

From 2024 CHIP-8 STM32-Port: CHIP-8 and S-CHIP emulator ported to STM32 Cortex M4 microcontroller.

From 2023 CHIP-8 Core: Core CHIP-8 and S-CHIP emulator library, used as the foundation for platform ports.

✧ Research Activities ✧

May 2026 **Committee Member** at the Int. Conf. on Software Language Engineering (SLE 2026), *ACM*, **B on CORE**

Dec 2025 **Reviewer** for the Eur. Conf. on Object Oriented Programming (ECOOP 2026), *ACM*, **A on CORE**

Nov 2025 **Reviewer** for the Journal of Computer Languages (COLA), *Elsevier*, **C on CORE**

Jun 2025 **Reviewer** for the Journal of Software and Systems Modeling (SoSyM), *Springer*, **Q1 on Scimago**

2–6 Jun 2025 **Participant** at <Programming>2025 conference

Apr–Jun 2025 **Committee Member** at the Int. Conf. on Software Language Engineering (SLE 2025), *ACM*, **B on CORE**

Mar 2025 **Reviewer** for the Journal of Systems and Software (JSS), *Elsevier*, **Q1 on Scimago**

Sep 2024 **Student Volunteer** at the Int. Conf. on Functional Programming (ICFP 2024)

✧ Dissemination Activities ✧

Jun 2026 A Multi-Dimensional, Per-Pass Empirical Study of the LLVM Optimization Pipeline — Ph.D. Student's Activity Report, University of Milan.

Mar 2026 MLIR: Scaling Compiler Infrastructure for Domain Specific Computation — Presentation of the paper by Lattner et al., 2021.

Feb 2026 Matheuristic Variants of DSATUR for the Vertex Coloring Problem — Presentation of the paper by Dupin, 2024.

Dec 2025 Your Optimizing Compiler is Not Optimizing Enough. To Hell With Multiple Recursions! — First MuseMI meeting of the 2025/2026 season, Milan.

Jul 2024 P4 Compiler in SDN — Presentation on the P4 compiler in Software Defined Networking.

Jul 2024 PhD Project — *Universal Language Server Protocol and Debugger Adapter Protocol for Modular Language Workbenches*, University of Milan.

Jul 2024 Master's Thesis — *Toward a Modular Approach for Type Systems and LSP Generation*, University of Milan.

✧ Teaching Activities ✧

Thesis Supervision

14/07/2026 M. Visconti, *Towards Neverlang3: A Generative Rust Approach for Resilient LL Parser Generators*, MSc, **102**

23/02/2026 D. Cerato, *Employing Metaprogramming: A Macro-Driven Strategy to Overcome Compiler-Imposed Limitations on Specialization*, BSc, **103**

10/04/2025 D. Pellegrino, *Scalable Multi-client Real-time Whisper*, BSc, **96**

09/04/2025 A. Longoni, *GUIDE: Graphical User Interface Development Environment*, MSc, **110L**

09/04/2025 L. Albani, *New Generalized Protocol For Software Product Line Extraction And Configuration*, MSc, **110L**

09/04/2025 G. Esposito, *Fr3D: A Framework for DAP-compatible DSL-oriented Debugging*, MSc, **110L**

24/02/2025 L. Favini, *RustyEx: Instrumenting* `rustc` *to Extract Feature Dependency Graphs*, BSc, **102**

Graduate Courses

2025–2026 Mathematical Logic (Art. 45), **BSc** in CS, University of Milan, *S. Aguzzoli*

2024–2025 Programming 1 (Art. 45), **BSc** in CS, University of Milan, *L. Capra* (coordinator W. Cazzola)

2024–2025 Mathematical Logic (Art. 45), **BSc** in CS, University of Milan, *S. Aguzzoli*

2023–2024 Mathematical Logic, **BSc** in CS, University of Milan, *S. Aguzzoli*

2023–2024 Computer Science, **BSc** in Comm. and Society, University of Milan, *A. Momigliano*

2023–2024 Programming 1, **BSc** in CS, University of Milan, *A. Trentini* (coordinator P. Boldi)

2023–2024 Programming in Python, **MSc** in Chemistry, University of Milan, *M. Monga*

Additional Activities

2023–Present **Private Tutoring** in Computer Science, Mathematics and Physics

Nov 2024 **Collaborator** at the Bebras Challenge, ALaDDIn Lab

Jan–Jun 2024 **Organizer** of the BSc Computer Science Laboratories, ALaDDIn Lab

Jan–Jun 2024 **Collaborator** of the Workshops for schools, ALaDDIn Lab

Nov 2023 **Collaborator** at the Bebras Challenge, ALaDDIn Lab

✧ Technical Skills ✧

Languages Rust, C/C++, Python, Go, OCaml, Java, Scala, Kotlin, Erlang, Lua, Dart, PHP, HTML/CSS, SQL, Bash, TeX

Systems/Tooling Git, CI/CD, Docker, GDB, Valgrind, Build Systems (e.g., CMake, Cargo, Pip), Cross-language Linking, Static/Dynamic Libraries, and FFI (e.g., C bindgen)

Area of Expertise Compiler Construction and Optimizations, Programming Languages Design, IR Design and Implementation, Type Systems, Language Support Tools (e.g., LSP), Static Analysis, Rustc Internals, MLIR, LLVM, Parsing Techniques and Parser Generators (e.g., ANTLR)

✧ Musical Activities ✧

2021–Present **Sound Engineer and Music Producer**, in collaboration with Believe and as a SIAE member.

2006–Present **Pianist and Music Composer**, composing original pieces and arrangements for piano and various ensembles

2019–Present **Piano and Music Teacher**, teaching piano and music theory and composition to students of all ages and levels

✧ Grants and Fellowships ✧

2024 56th Top Github Public Contributor in Italy out of 958

2023–2024 Scholarship for the MSc in Computer Science, awarded by the University of Milan

2020–2024 Scholarship for the BSc in Musical Computer Science, awarded by the University of Milan

✧ Tongues ✧

Italian Mother tongue

English Level CEFR B2 (SLAM at University of Milan)

Spanish Base (A1–A2)

Milan, October 19, 2026